

```
// =====
// PARTE 1: Imports, enums e declaração da classe (até variáveis)
// =====

import 'dart:async';
import 'dart:io';
import 'package:flutter/material.dart';
import 'package:gal/gal.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:google_mlkit_image_labeling/google_mlkit_image_labeling.dart';
import 'package:flutter_ffmpeg/flutter_ffmpeg.dart';
import '../controllers/microscopio_controller.dart';
import '../services/uvicamera_service.dart';
import '../services/armazenamento_fotos.dart';
import '../services/identificador_ia.dart';
import '../services/digitalizador_documento.dart';
import 'galeria_screen.dart';
import 'controles_screen.dart';
import 'anotacoes_tarefas_screen.dart';
import '../widgets/seletor_camera_usb_dialog.dart';

enum EfeitoCor { normal, invertido, pretoEBranco, falsaCor }

/// Ferramenta ativa para desenhar/anotar sobre a imagem da câmera.
enum FerramentaAtiva { nenhuma, desenho, texto }

/// Um texto colocado pelo usuário sobre a imagem, em coordenadas de tela.
class _AnotacaoTexto {
  Offset posicao;
  String texto;
  _AnotacaoTexto(this.posicao, this.texto);
}

class TelaPrincipal extends StatefulWidget {
  const TelaPrincipal({super.key});

  @override
  State<TelaPrincipal> createState() => _TelaPrincipalState();
}

class _TelaPrincipalState extends State<TelaPrincipal> {
  late final MicroscopioController _controller;
  double _flashOpacity = 0.0;
  String _feedback = "";
  List<File> _fotosSessao = [];
  final TransformationController _zoomController = TransformationController();

  bool _gravando = false;

```

```

String? _caminhoVideoAtual;

bool _modoMedicao = false;
Offset? _pontoA;
Offset? _pontoB;
double? _umPorPixel;

final IdentificadorIA _ia = IdentificadorIA();
bool _identificando = false;

EfeitoCor _efeitoCor = EfeitoCor.normal;

bool _uiVisivel = true;
bool _gradeAtiva = false;
bool _focoCentralAtivo = false;

// Ferramentas sobre a imagem: desenho à mão livre e texto.
FerramentaAtiva _ferramentaAtiva = FerramentaAtiva.nenhuma;
final List<List<Offset>> _tracosDesenho = [];
final List<_AnotacaoTexto> _anotacoesTexto = [];

// Slider de brilho flutuante sobre a imagem.
bool _brilhoSliderVisivel = false;
UvcCameraControl? _controleBrilho;
double? _valorBrilho;

// Dividir tela / comparar com uma foto salva.
File? _fotoComparacao;

// Time-lapse.
bool _timelapseAtivo = false;
int _timelapseIntervaloSegundos = 5;
int _timelapseContagem = 0;
Timer? _timelapseTimer;

bool _digitalizando = false;

// =====
// FIM DA PARTE 1
// =====//
=====
// PARTE 2: initState, conexão, carregamento, listener e captura/gravação
// =====

@override
void initState() {
  super.initState();
  _controller = MicroscopioController();

```

```

        _controller.addListener(_onControllerChanged);
        _iniciarConexao();
        _carregarFotosSalvas();
        _carregarCalibracao();
    }

    /// Verifica quantos dispositivos UVC estão conectados antes de abrir a
    /// câmera. Se houver mais de um, pergunta ao usuário qual usar.
    Future<void> _iniciarConexao() async {
        final dispositivos = await _controller.listarDispositivos();
        if (dispositivos.length > 1) {
            final escolhido = await mostrarSeletorCameraUsb(context, _controller, dispositivos);
            if (escolhido == null) {
                _mostrarFeedback('Seleção de câmera cancelada');
                return;
            }
            await _controller.inicializar(deviceId: escolhido);
        } else {
            await _controller.inicializar();
        }
    }

    /// Mesma lógica de seleção, usada pelo botão de reconectar.
    Future<void> _reconectar() async {
        final dispositivos = await _controller.listarDispositivos();
        if (dispositivos.length > 1) {
            final escolhido = await mostrarSeletorCameraUsb(context, _controller, dispositivos);
            if (escolhido == null) return;
            await _controller.reconectar(deviceId: escolhido);
        } else {
            await _controller.reconectar();
        }
    }

    Future<void> _carregarFotosSalvas() async {
        final arquivos = await ArmazenamentoFotos.listar();
        if (mounted) setState(() => _fotosSessao = arquivos);
    }

    Future<void> _carregarCalibracao() async {
        final prefs = await SharedPreferences.getInstance();
        final valor = prefs.getDouble('um_por_pixel');
        if (mounted && valor != null) {
            setState(() => _umPorPixel = valor);
        }
    }

    void _onControllerChanged() {

```

```

if (!mounted) return;
if (_timelapseAtivo && !_controller.estaConectado) {
    _pararTimelapse();
    return;
}
setState(() {});
}

```

```

Future<void> _capturar() async {
    final picture = _controller.capturarFoto();
    if (picture == null) {
        _mostrarFeedback('Falha ao capturar');
        return;
    }
    _dispararFlash();
    final arquivo = await ArmazenamentoFotos.salvar(picture.jpegBytes);
    setState(() => _fotosSessao.insert(0, arquivo));
    try {
        await Gal.putImageBytes(picture.jpegBytes, name:
'microscopio_${DateTime.now().millisecondsSinceEpoch}');
        _mostrarFeedback('Foto salva!');
    } on GalException catch (e) {
        _mostrarFeedback('Salva no app (erro na galeria: ${e.type.message})');
    }
}

```

```

Future<void> _alternarGravacao() async {
    if (_controller.estaGravando) {
        final codigo = _controller.pararGravacao();
        setState(() => _gravando = false);
        if (codigo == 0 && _caminhoVideoAtual != null) {
            try {
                await Gal.putVideo(_caminhoVideoAtual!);
                _mostrarFeedback('Vídeo salvo na galeria!');
            } on GalException catch (e) {
                _mostrarFeedback('Erro ao salvar vídeo: ${e.type.message}');
            }
        } else {
            _mostrarFeedback('Falha ao finalizar gravação');
        }
        return;
    }
    if (!_controller.estaConectado) {
        _mostrarFeedback('Conecte o microscópio primeiro');
        return;
    }
    final pasta = await ArmazenamentoFotos.pastaVideosTemp();
    final caminho = '${pasta.path}/video_${DateTime.now().millisecondsSinceEpoch}.mp4';
}

```

```

final codigo = _controller.iniciarGravacao(caminho);
if (codigo == 0) {
  setState(() {
    _gravando = true;
    _caminhoVideoAtual = caminho;
  });
} else {
  _mostrarFeedback('Falha ao iniciar gravação');
}
}

void _alternarModoMedicao() {
  setState(() {
    _modoMedicao = !_modoMedicao;
    _pontoA = null;
    _pontoB = null;
    if (_modoMedicao) {
      _ferramentaAtiva = FerramentaAtiva.nenhuma;
    }
  });
}

void _alternarUi() {
  setState(() => _uiVisivel = !_uiVisivel);
}

void _alternarGrade() {
  setState(() => _gradeAtiva = !_gradeAtiva);
}

void _alternarFocoCentral() {
  setState(() => _focoCentralAtivo = !_focoCentralAtivo);
}

// =====
// FIM DA PARTE 2
// =====//
=====
// PARTE 3: Ferramentas de desenho/texto, brilho, comparar e time-lapse
// =====

/// Ativa/desativa uma ferramenta (desenho ou texto). Tocar de novo na
/// ferramenta já ativa a desliga. Ativar uma desliga a outra e sai do
/// modo de medição, já que só uma ferramenta de toque funciona por vez.
void _selecionarFerramenta(FerramentaAtiva ferramenta) {
  setState(() {
    _ferramentaAtiva = _ferramentaAtiva == ferramenta ? FerramentaAtiva.nenhuma :
ferramenta;

```

```

    if (_ferramentaAtiva != FerramentaAtiva.nenhuma) {
        _modoMedicao = false;
    }
});
}

void _aoIniciarTraco(DragStartDetails details) {
    setState(() => _tracosDesenho.add([details.localPosition]));
}

void _aoContinuarTraco(DragUpdateDetails details) {
    if (_tracosDesenho.isEmpty) return;
    setState(() => _tracosDesenho.last.add(details.localPosition));
}

Future<void> _aoTocarParaTexto(TapDownDetails details) async {
    final posicao = details.localPosition;
    final controladorTexto = TextEditingController();
    final texto = await showDialog<String>(
        context: context,
        builder: (context) => AlertDialog(
            title: const Text('Adicionar texto'),
            content: TextField(
                controller: controladorTexto,
                autofocus: true,
                decoration: const InputDecoration(labelText: 'Texto sobre a imagem'),
            ),
            actions: [
                TextButton(onPressed: () => Navigator.pop(context), child: const Text('Cancelar')),
                TextButton(
                    onPressed: () => Navigator.pop(context, controladorTexto.text),
                    child: const Text('Adicionar'),
                ),
            ],
        ),
    );
    if (texto != null && texto.trim().isNotEmpty) {
        setState(() => _anotacoesTexto.add(_AnotacaoTexto(posicao, texto.trim())));
    }
}

/// Apaga todos os desenhos e textos colocados sobre a imagem. Não afeta
/// medições, grade ou foco central, que têm seus próprios controles.
void _limparAnotacoes() {
    if (_tracosDesenho.isEmpty && _anotacoesTexto.isEmpty) return;
    setState(() {
        _tracosDesenho.clear();
        _anotacoesTexto.clear();
    });
}

```

```

});
_mostrarFeedback('Anotações apagadas');
}

/// Procura, entre os controles UVC disponíveis, o que corresponde ao
/// brilho da imagem (label contendo "brightness" ou "brilho").
UvcCameraControl? _encontrarControleBrilho() {
    final controles = _controller.controlesDisponiveis();
    for (final c in controles) {
        final label = c.label.toLowerCase();
        if (label.contains('brightness') || label.contains('brilho')) {
            return c;
        }
    }
    return null;
}

void _alternarBrilhoSlider() {
    if (_brilhoSliderVisivel) {
        setState(() => _brilhoSliderVisivel = false);
        return;
    }
    final controle = _encontrarControleBrilho();
    if (controle == null) {
        _mostrarFeedback('Este microscópio não expõe controle de brilho via USB');
        return;
    }
    final atual = _controller.lerControle(controle.id) ?? controle.min;
    setState(() {
        _controleBrilho = controle;
        _valorBrilho = atual.toDouble();
        _brilhoSliderVisivel = true;
    });
}

void _aoMudarBrilho(double valor) {
    setState(() => _valorBrilho = valor);
    _controller.definirControle(_controleBrilho!.id, valor.round());
}

/// Liga/desliga o modo "dividir tela": mostra a câmera ao vivo de um
/// lado e uma foto já salva do outro, para comparação lado a lado.
Future<void> _alternarComparar() async {
    if (_fotoComparacao != null) {
        setState(() => _fotoComparacao = null);
        return;
    }
    if (_fotosSessao.isEmpty) {

```

```

    _mostrarFeedback('Nenhuma foto salva para comparar ainda');
    return;
}
final escolhida = await showModalBottomSheet<File>(
  context: context,
  backgroundColor: Colors.black87,
  isScrollControlled: true,
  builder: (context) {
    return SafeArea(
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Text(
              'Escolha uma foto para comparar',
              style: TextStyle(color: Colors.white, fontSize: 16, fontWeight: FontWeight.bold),
            ),
            const SizedBox(height: 12),
            SizedBox(
              height: 220,
              child: GridView.builder(
                gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
                  crossAxisCount: 3,
                  crossAxisSpacing: 8,
                  mainAxisSpacing: 8,
                ),
                itemCount: _fotosSessao.length,
                itemBuilder: (context, index) {
                  final foto = _fotosSessao[index];
                  return GestureDetector(
                    onTap: () => Navigator.pop(context, foto),
                    child: ClipRRect(
                      borderRadius: BorderRadius.circular(8),
                      child: Image.file(foto, fit: BoxFit.cover),
                    ),
                  ),
                );
              ),
            ),
          ],
        ),
      );
    },
  );
  if (escolhida != null) {

```

```

    setState(() => _fotoComparacao = escolhida);
  }
}

```

/// Abre a configuração de time-lapse (intervalo entre fotos) e, se
/// confirmado, inicia a captura periódica.

```
Future<void> _abrirConfigTimelapse() async {
  if (_timelapseAtivo) {
    _pararTimelapse();
    return;
  }
  if (!_controller.estaConectado) {
    _mostrarFeedback('Conecte o microscópio primeiro');
    return;
  }
  var intervaloEscolhido = _timelapseIntervaloSegundos;
  final confirmou = await showModalBottomSheet<bool>(
    context: context,
    backgroundColor: Colors.black87,
    builder: (context) {
      return StatefulBuilder(
        builder: (context, setModalState) {
          return SafeArea(
            child: Padding(
              padding: const EdgeInsets.all(20),
              child: Column(
                mainAxisAlignment: MainAxisAlignment.start,
                children: [
                  const Text(
                    'Iniciar time-lapse',
                    style: TextStyle(color: Colors.white, fontSize: 18, fontWeight: FontWeight.bold),
                  ),
                  const SizedBox(height: 4),
                  const Text(
                    'Uma foto será capturada automaticamente a cada intervalo escolhido.',
                    style: TextStyle(color: Colors.white54, fontSize: 12),
                  ),
                  const SizedBox(height: 16),
                  Wrap(
                    spacing: 8,
                    children: [5, 10, 30, 60].map((segundos) {
                      return ChoiceChip(
                        label: Text('${segundos}s'),
                        selected: intervaloEscolhido == segundos,
                        onSelected: (_) => setModalState(() => intervaloEscolhido = segundos),
                      );
                    }).toList(),
                  ),
                ],
              ),
            ),
          ),
        ),
      ),
    ),
  );
}
```

```

    ),
    const SizedBox(height: 20),
    ElevatedButton(
      onPressed: () => Navigator.pop(context, true),
      child: const Text('Iniciar'),
    ),
  ],
),
),
);
},
);
},
);
);
if (confirmou == true) {
  setState(() {
    _timelapseIntervaloSegundos = intervaloEscolhido;
    _timelapseAtivo = true;
    _timelapseContagem = 0;
  });
  _timelapseTimer = Timer.periodic(
    Duration(seconds: _timelapseIntervaloSegundos),
    (_) => _capturarQuadroTimelapse(),
  );
}
}

Future<void> _capturarQuadroTimelapse() async {
  final picture = _controller.capturarFoto();
  if (picture == null) return;
  final arquivo = await ArmazenamentoFotos.salvar(picture.jpegBytes);
  if (!mounted) return;
  setState(() {
    _fotosSessao.insert(0, arquivo);
    _timelapseContagem++;
  });
}

void _pararTimelapse() {
  _timelapseTimer?.cancel();
  _timelapseTimer = null;
  setState(() => _timelapseAtivo = false);
  _mostrarFeedback('Time-lapse parado — $_timelapseContagem fotos capturadas');
}

// =====
// FIM DA PARTE 3

```

```
// =====//
// PARTE 4: Digitalização, ejeta, ajuda, ocultar UI, medição/calibração
// =====
```

```
/// Captura a imagem atual e salva uma versão "digitalizada" (preto e
/// branco, mais contraste), útil para anotações escritas sobre a lâmina.
Future<void> _digitalizar() async {
  if (!_controller.estaConectado) {
    _mostrarFeedback('Conecte o microscópio primeiro');
    return;
  }
  final picture = _controller.capturarFoto();
  if (picture == null) {
    _mostrarFeedback('Falha ao capturar imagem');
    return;
  }
  setState(() => _digitalizando = true);
  try {
    final processada = DigitalizadorDocumento.digitalizar(picture.jpegBytes);
    if (processada == null) {
      _mostrarFeedback('Falha ao digitalizar');
      return;
    }
    final arquivo = await ArmazenamentoFotos.salvar(processada);
    setState(() => _fotosSessao.insert(0, arquivo));
    _mostrarFeedback('Digitalização salva!');
  } finally {
    if (mounted) setState(() => _digitalizando = false);
  }
}
```

```
/// Ejeta o microscópio: apenas desconecta, sem tentar reconectar
/// sozinho (diferente do botão de reconectar, que já existe).
```

```
Future<void> _ejeta() async {
  if (_timelapseAtivo) _pararTimelapse();
  await _controller.ejeta();
  _mostrarFeedback('Microscópio ejetado');
}
```

```
void _mostrarAjuda() {
  Widget item(IconData icon, String texto) {
    return Padding(
      padding: const EdgeInsets.only(bottom: 12),
      child: Row(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Icon(icon, color: Colors.white70, size: 20),
```

```

        const SizedBox(width: 12),
        Expanded(child: Text(texto, style: const TextStyle(color: Colors.white, fontSize: 13))),
      ],
    ),
  );
}

```

```

showModalBottomSheet(
  context: context,
  backgroundColor: Colors.black87,
  isScrollControlled: true,
  builder: (context) {
    return SafeArea(
      child: SingleChildScrollView(
        padding: const EdgeInsets.all(20),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Text(
              'Ajuda rápida',
              style: TextStyle(color: Colors.white, fontSize: 18, fontWeight: FontWeight.bold),
            ),
            const SizedBox(height: 16),
            item(Icons.camera_alt, 'Capturar foto'),
            item(Icons.videocam, 'Gravar vídeo'),
            item(Icons.straighten, 'Modo de medição (toque dois pontos na imagem)'),
            item(Icons.zoom_in, 'Redefinir zoom'),
            item(Icons.refresh, 'Reconectar ao microscópio'),
            item(Icons.eject, 'Ejetar: desconecta sem reconectar sozinho'),
            item(Icons.grid_on, 'Grade de composição (regra dos terços)'),
            item(Icons.center_focus_weak, 'Mira de foco central'),
            item(Icons.title, 'Adicionar texto sobre a imagem'),
            item(Icons.edit, 'Desenho à mão livre sobre a imagem'),
            item(Icons.delete_outline, 'Apagar desenhos e textos'),
            item(Icons.brightness_6, 'Slider de brilho da câmera'),
            item(Icons.vertical_split, 'Dividir tela para comparar com uma foto salva'),
            item(Icons.timelapse, 'Time-lapse: captura fotos automaticamente em intervalos'),
            item(Icons.document_scanner, 'Digitalizar: salva uma versão em preto e branco
de alto contraste'),
            item(Icons.auto_awesome, 'Identificar com IA (reconhecimento genérico)'),
            item(Icons.photo_library, 'Abrir galeria de fotos'),
            item(Icons.lightbulb_outline, 'Ligar/desligar luz do microscópio'),
            item(Icons.palette, 'Efeitos de cor'),
            item(Icons.checklist_outlined, 'Anotações & Tarefas'),
            item(Icons.settings, 'Configurações'),
            const SizedBox(height: 4),
            const Text(
              'Toque na tela para esconder/mostrar os botões.',

```

```

        style: TextStyle(color: Colors.white54, fontSize: 12, fontStyle: FontStyle.italic),
      ),
    ],
  ),
),
);
},
);
}

```

/// Envolve widgets de UI (barras de botões) que devem sumir ao tocar na
 /// tela. A coluna lateral esquerda não usa isso: ela fica sempre fixa.

```

Widget _ocultavel(Widget child) {
  return IgnorePointer(
    ignoring: !_uiVisivel,
    child: AnimatedOpacity(
      opacity: _uiVisivel ? 1.0 : 0.0,
      duration: const Duration(milliseconds: 200),
      child: child,
    ),
  );
}

```

```

void _aoTocarParaMedicao(TapDownDetails details) {
  setState(() {
    if (_pontoA == null || _pontoB != null) {
      _pontoA = details.localPosition;
      _pontoB = null;
    } else {
      _pontoB = details.localPosition;
    }
  });
}

```

```

double? get _distanciaPixels {
  if (_pontoA == null || _pontoB == null) return null;
  return (_pontoB! - _pontoA!).distance;
}

```

```

String _textoDistancia() {
  final px = _distanciaPixels!;
  if (_umPorPixel != null) {
    final um = px * _umPorPixel!;
    if (um >= 1000) {
      return '${(um / 1000).toStringAsFixed(2)} mm';
    }
    return '${um.toStringAsFixed(1)} µm';
  }
}

```

```
    return '${px.toStringAsFixed(0)} px (não calibrado)';
  }
```

```
Future<void> _calibrar() async {
  final controladorTexto = TextEditingController();
  final resultado = await showDialog<double>(
    context: context,
    builder: (context) => AlertDialog(
      title: const Text('Calibrar medição'),
      content: TextField(
        controller: controladorTexto,
        keyboardType: const TextInputType.numberWithOptions(decimal: true),
        decoration: const InputDecoration(labelText: 'Distância real entre os pontos (µm)'),
      ),
      actions: [
        TextButton(onPressed: () => Navigator.pop(context), child: const Text('Cancelar')),
        TextButton(
          onPressed: () {
            final valor = double.tryParse(controladorTexto.text.replaceAll(',', '.'));
            Navigator.pop(context, valor);
          },
          child: const Text('Salvar'),
        ),
      ],
    ),
  );
  if (resultado != null && resultado > 0 && _distanciaPixels != null) {
    final fator = resultado / _distanciaPixels!;
    final prefs = await SharedPreferences.getInstance();
    await prefs.setDouble('um_por_pixel', fator);
    setState(() => _umPorPixel = fator);
    _mostrarFeedback('Calibração salva!');
  }
}
```

```
// =====
// FIM DA PARTE 4
// =====
=====
// PARTE 5: IA, formato de modo, seletor de modo, reset zoom, luz, efeitos
// =====
```

```
Future<void> _identificar() async {
  if (!_controller.estaConectado) {
    _mostrarFeedback('Conecte o microscópio primeiro');
    return;
  }
  final picture = _controller.capturarFoto();
```

```

if (picture == null) {
    _mostrarFeedback('Falha ao capturar imagem para análise');
    return;
}
setState(() => _identificando = true);
try {
    final pasta = await ArmazenamentoFotos.pastaVideosTemp();
    final arquivoTemp =
File('${pasta.path}/analise_${DateTime.now().millisecondsSinceEpoch}.jpg');
    await arquivoTemp.writeAsBytes(picture.jpegBytes);
    final resultados = await _ia.identificar(arquivoTemp);
    if (mounted) _mostrarResultadosIA(resultados);
    await arquivoTemp.delete();
} catch (e) {
    _mostrarFeedback('Falha na identificação');
} finally {
    if (mounted) setState(() => _identificando = false);
}
}

void _mostrarResultadosIA(List<ImageLabel> resultados) {
    showModalBottomSheet(
        context: context,
        backgroundColor: Colors.black87,
        builder: (context) {
            return Padding(
                padding: const EdgeInsets.all(20),
                child: Column(
                    mainAxisAlignment: MainAxisAlignment.min,
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                        const Text(
                            'O que a IA identificou',
                            style: TextStyle(color: Colors.white, fontSize: 18, fontWeight: FontWeight.bold),
                        ),
                        const SizedBox(height: 4),
                        const Text(
                            'Reconhecimento genérico local, não especializado em microscopia.',
                            style: TextStyle(color: Colors.white54, fontSize: 12),
                        ),
                        const SizedBox(height: 16),
                        if (resultados.isEmpty)
                            const Text(
                                'Nada reconhecido com confiança suficiente.',
                                style: TextStyle(color: Colors.white70),
                            )
                        else
                            ...resultados.map(

```

```

(r) => Padding(
  padding: const EdgeInsets.only(bottom: 10),
  child: Row(
    children: [
      Expanded(
        child: Text(
          r.label,
          style: const TextStyle(color: Colors.white, fontSize: 15),
        ),
      ),
      Text(
        '${(r.confidence * 100).toStringAsFixed(0)}%',
        style: const TextStyle(color: Colors.white70),
      ),
    ],
  ),
),
),
),
),
],
),
),
);
},
);
}

```

```

/// Tenta identificar o formato (codec) do modo a partir do texto do
/// `label` retornado pelo plugin UVC (ex.: "MJPEG", "YUYV", "YUY2"...).
/// Retorna null se não reconhecer nenhum formato conhecido no texto.
String? _formatoDoModo(UvcCameraMode modo) {
  final texto = modo.label.toUpperCase();
  const formatosConhecidos = ['MJPEG', 'MJPEG', 'YUYV', 'YUY2', 'UYVY', 'NV12', 'H264',
'RGB565'];
  for (final formato in formatosConhecidos) {
    if (texto.contains(formato)) {
      return formato == 'MJPEG' ? 'MJPEG' : formato;
    }
  }
  return null;
}

```

```

/// Selo colorido indicando o formato do modo de câmera. MJPEG (comprimido,
/// costuma liberar resoluções maiores) aparece em verde; formatos brutos
/// como YUYV/YUY2 aparecem em âmbar.
Widget _seloFormato(String formato) {
  final comprimido = formato == 'MJPEG' || formato == 'H264';
  final cor = comprimido ? Colors.greenAccent : Colors.amberAccent;
  return Container(
    padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 2),

```

```

decoration: BoxDecoration(
  color: cor.withValues(alpha: 0.15),
  borderRadius: BorderRadius.circular(20),
  border: Border.all(color: cor.withValues(alpha: 0.6)),
),
child: Text(
  formato,
  style: TextStyle(color: cor, fontSize: 10.5, fontWeight: FontWeight.w700),
),
);
}

```

```

void _abrirSeletorModo() {
  final modos = _controller.modosDisponiveis();
  if (modos.isEmpty) {
    _mostrarFeedback('Conecte o microscópio para ver os modos disponíveis');
    return;
  }
  final modoAtual = _controller.modoAtual;
  showModalBottomSheet(
    context: context,
    backgroundColor: Colors.black87,
    isScrollControlled: true,
    builder: (context) {
      return SafeArea(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            const Padding(
              padding: EdgeInsets.fromLTRB(16, 16, 16, 4),
              child: Text(
                'Modo de câmera',
                style: TextStyle(color: Colors.white, fontSize: 18, fontWeight: FontWeight.bold),
              ),
            ),
            const Padding(
              padding: EdgeInsets.fromLTRB(16, 0, 16, 12),
              child: Text(
                'MJPEG é comprimido e costuma liberar resoluções maiores. YUYV é bruto (sem compressão) e por isso fica mais limitado.',
                style: TextStyle(color: Colors.white38, fontSize: 11.5),
              ),
            ),
            Flexible(
              child: ListView.builder(
                shrinkWrap: true,
                itemCount: modos.length,
                itemBuilder: (context, index) {

```

```

final modo = modos[index];
final selecionado = modoAtual != null &&
    modo.width == modoAtual.width &&
    modo.height == modoAtual.height &&
    modo.fps == modoAtual.fps &&
    modo.label == modoAtual.label;
final formato = _formatoDoModo(modo);
return ListTile(
    leading: Icon(
        selecionado ? Icons.radio_button_checked : Icons.radio_button_off,
        color: selecionado ? Colors.greenAccent : Colors.white54,
    ),
    title: Row(
        children: [
            Text(
                '${modo.width} × ${modo.height}',
                style: const TextStyle(color: Colors.white, fontSize: 15),
            ),
            if (formato != null) ...[
                const SizedBox(width: 8),
                _seloFormato(formato),
            ],
        ],
    ),
    subtitle: Text(
        '${modo.label} · ${modo.fps} fps',
        style: const TextStyle(color: Colors.white54, fontSize: 12),
    ),
    onTap: () async {
        Navigator.pop(context);
        _mostrarFeedback('Trocando modo...');
        final ok = await _controller.selecionarModo(modo);
        _mostrarFeedback(ok ? 'Modo alterado' : 'Falha ao trocar de modo');
    },
);
},
),
),
],
),
);
},
);
}

void _resetarZoom() {
    _zoomController.value = Matrix4.identity();
}

```

```

void _alternarLuz() {
  final controles = _controller.controlesDisponiveis();
  UvcCameraControl? controleLuz;
  for (final c in controles) {
    final label = c.label.toLowerCase();
    if (label.contains('backlight') || label.contains('luz') || label.contains('light')) {
      controleLuz = c;
      break;
    }
  }
  if (controleLuz == null) {
    _mostrarFeedback('Este microscópio não expõe controle de luz via USB');
    return;
  }
  final atual = _controller.lerControle(controleLuz.id) ?? controleLuz.min;
  final novoValor = atual > controleLuz.min ? controleLuz.min : controleLuz.max;
  final ok = _controller.definirControle(controleLuz.id, novoValor);
  _mostrarFeedback(ok ? 'Luz ajustada' : 'Falha ao ajustar a luz');
}

```

```

void _abrirSeletorEfeitos() {
  showModalBottomSheet(
    context: context,
    backgroundColor: Colors.black87,
    builder: (context) {
      Widget opcao(String label, EfeitoCor efeito) {
        return ListTile(
          title: Text(label, style: const TextStyle(color: Colors.white)),
          trailing: _efeitoCor == efeito
            ? const Icon(Icons.check, color: Colors.white)
            : null,
          onTap: () {
            setState(() => _efeitoCor = efeito);
            Navigator.pop(context);
          },
        );
      }
    },
  );
}

```

```

return SafeArea(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.min,
    children: [
      opcao('Normal', EfeitoCor.normal),
      opcao('Invertido', EfeitoCor.invertido),
      opcao('Preto e branco', EfeitoCor.pretoEBranco),
      opcao('Falsa cor', EfeitoCor.falsaCor),
    ],
  ),
)

```

```

    ),
  );
},
);
}

```

```

ColorFilter? _filtroEfeitoAtual() {
  switch (_efeitoCor) {
    case EfeitoCor.normal:
      return null;
    case EfeitoCor.invertido:
      return const ColorFilter.matrix(<double>[
        -1, 0, 0, 0, 255,
        0, -1, 0, 0, 255,
        0, 0, -1, 0, 255,
        0, 0, 0, 1, 0,
      ]);
    case EfeitoCor.pretoEBranco:
      return const ColorFilter.matrix(<double>[
        0.2126, 0.7152, 0.0722, 0, 0,
        0.2126, 0.7152, 0.0722, 0, 0,
        0.2126, 0.7152, 0.0722, 0, 0,
        0, 0, 0, 1, 0,
      ]);
    case EfeitoCor.falsaCor:
      return const ColorFilter.matrix(<double>[
        0, 0, 1.4, 0, 0,
        0, 1.1, 0, 0, 0,
        1.4, 0, 0, 0, 0,
        0, 0, 0, 1, 0,
      ]);
  }
}

```

```

// =====
// FIM DA PARTE 5
// =====//
=====
// PARTE 6: Navegação, status, flash, feedback, dispose e build (início)
// =====

```

```

void _abrirGaleria() {
  Navigator.push(
    context,
    MaterialPageRoute(builder: (_) => GaleriaScreen(fotos: _fotosSessao)),
  );
}

```

```

void _abrirAnotacoesTarefas() {
  Navigator.push(
    context,
    MaterialPageRoute(builder: (_) => const AnotacoesTarefasScreen()),
  );
}

void _abrirControles() {
  Navigator.push(
    context,
    MaterialPageRoute(builder: (_) => ControlesScreen(controller: _controller)),
  );
}

Color _corStatus() {
  switch (_controller.status) {
    case UvcStatus.connected:
      return Colors.green;
    case UvcStatus.connecting:
      return Colors.orange;
    case UvcStatus.error:
      return Colors.red;
    case UvcStatus.disconnected:
      return Colors.grey;
  }
}

void _dispararFlash() {
  setState(() => _flashOpacity = 1.0);
  Future.delayed(const Duration(milliseconds: 180), () {
    if (mounted) setState(() => _flashOpacity = 0.0);
  });
}

void _mostrarFeedback(String texto) {
  setState(() => _feedback = texto);
  Future.delayed(const Duration(seconds: 1), () {
    if (mounted) setState(() => _feedback = "");
  });
}

@override
void dispose() {
  _controller.removeListener(_onControllerChanged);
  _controller.dispose();
  _zoomController.dispose();
  _ja.dispose();
  _timelapseTimer?.cancel();
}

```

```
    super.dispose();  
  }
```

```
@override
```

```
Widget build(BuildContext context) {  
  return Scaffold(  
    backgroundColor: Colors.black,  
    body: Stack(  
      children: [  
        Positioned.fill(child: _buildPreview()),  
        if (_gradeAtiva)  
          const Positioned.fill(  
            child: IgnorePointer(  
              child: CustomPaint(painter: _GradePainter()),  
            ),  
          ),  
        if (_focoCentralAtivo)  
          const Positioned.fill(  
            child: IgnorePointer(  
              child: CustomPaint(painter: _FocoCentralPainter()),  
            ),  
          ),  
        IgnorePointer(  
          child: AnimatedOpacity(  
            opacity: _flashOpacity,  
            duration: const Duration(milliseconds: 80),  
            child: Container(color: Colors.white),  
          ),  
        ),  
        if (_feedback.isNotEmpty)  
          Center(  
            child: Container(  
              padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 12),  
              decoration: BoxDecoration(  
                color: Colors.black87,  
                borderRadius: BorderRadius.circular(12),  
              ),  
            child: Text(_feedback, style: const TextStyle(color: Colors.white, fontSize: 16)),  
          ),  
        ),  
        if (_modoMedicao)  
          Positioned.fill(  
            child: GestureDetector(  
              onTapDown: _aoTocarParaMedicao,  
              child: CustomPaint(  
                painter: _MedicaoPainter(_pontoA, _pontoB),  
                size: Size.infinite,  
              ),  
            ),  
          ),  
      ],  
    ),  
  );  
}
```

```
,
),
// Traços de desenho à mão livre e textos, sempre visíveis mesmo
// quando a ferramenta não está ativa (para continuar vendo o que
// já foi desenhado enquanto navega pela imagem).
if (_tracosDesenho.isNotEmpty || _anotacoesTexto.isNotEmpty)
  Positioned.fill(
    child: IgnorePointer(
      child: CustomPaint(painter: _DesenhoPainter(_tracosDesenho,
        _anotacoesTexto)),
    ),
  ),
),
if (_ferramentaAtiva == FerramentaAtiva.desenho)
  Positioned.fill(
    child: GestureDetector(
      onTapStart: _aoIniciarTraco,
      onTapUpdate: _aoContinuarTraco,
    ),
  ),
),
if (_ferramentaAtiva == FerramentaAtiva.texto)
  Positioned.fill(
    child: GestureDetector(
      onTapDown: _aoTocarParaTexto,
    ),
  ),
),
if (_brilhoSliderVisivel && _controleBrilho != null)
  Positioned(
    bottom: 96,
    left: 24,
    right: 24,
    child: Container(
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 4),
      decoration: BoxDecoration(
        color: Colors.black87,
        borderRadius: BorderRadius.circular(24),
      ),
    ),
    child: Row(
      children: [
        const Icon(Icons.brightness_6, color: Colors.white70, size: 20),
        Expanded(
          child: Slider(
            value: _valorBrilho!.clamp(
              _controleBrilho!.min.toDouble(),
              _controleBrilho!.max.toDouble(),
            ),
            min: _controleBrilho!.min.toDouble(),
            max: _controleBrilho!.max.toDouble(),
            onChanged: aoMudarBrilho,
```

```

        ),
    ),
],
),
),
),
Positioned(
  top: 16,
  left: 16,
  child: _ocultavel(
    Container(
      width: 14,
      height: 14,
      decoration: BoxDecoration(
        shape: BoxShape.circle,
        color: _corStatus(),
        border: Border.all(color: Colors.white, width: 1.5),
      ),
    ),
  ),
),
),
// Coluna esquerda: câmera, vídeo, régua, zoom, atualizar, ejetar
Positioned(
  left: 16,
  top: 0,
  bottom: 0,
  child: Center(
    child: SingleChildScrollView(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          _botaoTopo(Icons.camera_alt, _capturar),
          const SizedBox(height: 10),
          _botaoTopo(
            _gravando ? Icons.stop : Icons.videocam,
            _alternarGravacao,
            ativo: _gravando,
          ),
          const SizedBox(height: 10),
          _botaoTopo(Icons.straighten, _alternarModoMedicao, ativo: _modoMedicao),
          const SizedBox(height: 10),
          _botaoTopo(Icons.zoom_in, _resetarZoom),
          const SizedBox(height: 10),
          _botaoTopo(Icons.refresh, _reconectar),
          const SizedBox(height: 10),
          _botaoTopo(Icons.eject, _ejetar),
          const SizedBox(height: 10),
          _botaoTopo(

```

```

        _gradeAtiva ? Icons.grid_on : Icons.grid_off,
        _alternarGrade,
        ativo: _gradeAtiva,
    ),
    const SizedBox(height: 10),
    _botaoTopo(Icons.center_focus_weak, _alternarFocoCentral, ativo:
    _focoCentralAtivo),
    ],
    ),
    ),
    ),
    ),
    // Coluna direita: galeria, luz, cores, configurações
    Positioned(
      right: 16,
      top: 16,
      bottom: 16,
      child: _ocultavel(
        SingleChildScrollView(
          child: Column(
            children: [
              _botaoTopo(Icons.help_outline, _mostrarAjuda),
              const SizedBox(height: 10),
              _botaoTopo(Icons.photo_library, _abrirGaleria),
              const SizedBox(height: 10),
              _botaoTopo(Icons.lightbulb_outline, _alternarLuz),
              const SizedBox(height: 10),
              _botaoTopo(Icons.palette, _abrirSeletorEfeitos),
              const SizedBox(height: 10),
              _botaoTopo(Icons.checklist_outlined, _abrirAnotacoesTarefas),
              const SizedBox(height: 10),
              _botaoTopo(Icons.settings, _abrirControles),
            ],
          ),
        ),
      ),
    ),
    // Barra secundária: girar, espelhar, identificar com IA
    Positioned(
      top: 16,
      left: 0,
      right: 0,
      child: Center(
        child: _ocultavel(
          Row(
            mainAxisAlignment: MainAxisAlignment.min,
            children: [
              _botaoTopo(Icons.rotate_left, _controller.girarAntiHorario),

```

```

const SizedBox(width: 8),
_botaoTopo(Icons.rotate_right, _controller.girarHorario),
const SizedBox(width: 8),
_botaoTopo(Icons.flip, _controller.espelharHorizontal),
const SizedBox(width: 8),
_botaoTopo(Icons.swap_vert, _controller.espelharVertical),
const SizedBox(width: 8),
_chipModo(),
const SizedBox(width: 8),
_identificando
? Container(
  width: 40,
  height: 40,
  padding: const EdgeInsets.all(10),
  decoration: const BoxDecoration(shape: BoxShape.circle, color:
Colors.black45),
  child: const CircularProgressIndicator(strokeWidth: 2, color: Colors.white),
)
: _botaoTopo(Icons.auto_awesome, _identificar),
],
),
),
),
),
// Barra flutuante de funções adicionais: dividir tela, time-lapse
// e digitalizar.
Positioned(
  bottom: 76,
  left: 0,
  right: 0,
  child: Center(
    child: _ocultavel(
      Row(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          _botaoTopo(Icons.vertical_split, _alternarComparar, ativo: _fotoComparacao !=
null),
          const SizedBox(width: 8),
          _botaoTopo(Icons.timelapse, _abrirConfigTimelapse, ativo: _timelapseAtivo),
          const SizedBox(width: 8),
          _digitalizando
? Container(
  width: 40,
  height: 40,
  padding: const EdgeInsets.all(10),
  decoration: const BoxDecoration(shape: BoxShape.circle, color:
Colors.black45),
  child: const CircularProgressIndicator(strokeWidth: 2, color: Colors.white)

```

```

        )
        : _botaoTopo(Icons.document_scanner, _digitalizar),
    ],
  ),
),
),
),
// Barra flutuante de ferramentas sobre a imagem: texto, desenho,
// apagar e brilho.
Positioned(
  bottom: 24,
  left: 0,
  right: 0,
  child: Center(
    child: _ocultavel(
      Row(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          _botaoTopo(
            Icons.title,
            () => _selecionarFerramenta(FerramentaAtiva.texto),
            ativo: _ferramentaAtiva == FerramentaAtiva.texto,
          ),
          const SizedBox(width: 8),
          _botaoTopo(
            Icons.edit,
            () => _selecionarFerramenta(FerramentaAtiva.desenho),
            ativo: _ferramentaAtiva == FerramentaAtiva.desenho,
          ),
          const SizedBox(width: 8),
          _botaoTopo(Icons.delete_outline, _limparAnotacoes),
          const SizedBox(width: 8),
          _botaoTopo(Icons.brightness_6, _alternarBrilhoSlider, ativo:
            _brilhoSliderVisivel),
        ],
      ),
    ),
  ),
),
),
),
// =====
// FIM DA PARTE 6 (continua na parte 7 com o restante do build)
// =====
// =====
// PARTE 7: Restante do build (indicadores) e métodos auxiliares
// =====

    if (_timelapseAtivo)
      Positioned(

```

```

top: 44,
left: 0,
right: 0,
child: Center(
  child: Container(
    padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 8),
    decoration: BoxDecoration(
      color: Colors.black87,
      borderRadius: BorderRadius.circular(20),
    ),
  ),
  child: Row(
    mainAxisAlignment: MainAxisAlignment.min,
    children: [
      const Icon(Icons.timelapse, color: Colors.redAccent, size: 16),
      const SizedBox(width: 6),
      Text(
        'Time-lapse ativo · $_timelapseContagem fotos',
        style: const TextStyle(color: Colors.white, fontSize: 12.5),
      ),
    ],
  ),
),
),
),
),
),
if (_timelapseAtivo)
  Positioned(
    bottom: 130,
    left: 0,
    right: 0,
    child: Center(
      child: Container(
        padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 6),
        decoration: BoxDecoration(
          color: Colors.black87,
          borderRadius: BorderRadius.circular(20),
        ),
      ),
      child: Row(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          Text(
            'A cada ${_timelapseIntervaloSegundos}s',
            style: const TextStyle(color: Colors.white70, fontSize: 12),
          ),
          const SizedBox(width: 12),
          TextButton(
            onPressed: _pararTimelapse,
            style: TextButton.styleFrom(padding: EdgeInsets.zero, minimumSize: const
Size(0, 0)),

```



```

        SizedBox(height: 16),
        Text('Conectando ao microscópio...', style: TextStyle(color: Colors.white70)),
      ],
    ),
  );
}
if (_controller.estaConectado && _controller.textureId != null) {
  final ferramentaDeToqueAtiva = _modoMedicao || _ferramentaAtiva !=
FerramentaAtiva.nenhuma;
  final aoVivo = GestureDetector(
    behavior: HitTestBehavior.opaque,
    onTap: ferramentaDeToqueAtiva ? null : _alternarUi,
    onDoubleTap: ferramentaDeToqueAtiva ? null : _resetarZoom,
    child: Center(
      child: InteractiveViewer(
        transformationController: _zoomController,
        minScale: 1.0,
        maxScale: 5.0,
        panEnabled: !ferramentaDeToqueAtiva,
        scaleEnabled: !ferramentaDeToqueAtiva,
        child: AspectRatio(
          aspectRatio: _controller.currentAspectRatio ?? 16 / 9,
          child: ColorFiltered(
            colorFilter: _filtroEfeitoAtual() ?? const ColorFilter.mode(Colors.transparent,
BlendMode.dst),
            child: Texture(textureId: _controller.textureId!),
          ),
        ),
      ),
    ),
  );
if (_fotoComparacao == null) return aoVivo;
// Modo dividir tela: ao vivo à esquerda, foto salva à direita.
return Row(
  children: [
    Expanded(child: aoVivo),
    Container(width: 1, color: Colors.white24),
    Expanded(
      child: Stack(
        fit: StackFit.expand,
        children: [
          Image.file(_fotoComparacao!, fit: BoxFit.contain),
          const Positioned(
            top: 8,
            left: 8,
            child: Text('Foto salva', style: TextStyle(color: Colors.white70, fontSize: 11)),
          ),
        ],
      ),
    ),
  ],
);

```

```

    ),
  ),
],
);
}
return Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.min,
    children: [
      const Icon(Icons.usb_off, size: 64, color: Colors.white38),
      const SizedBox(height: 16),
      Text(
        _controller.lastError ?? 'Conecte o microscópio USB',
        style: const TextStyle(color: Colors.white70, fontSize: 16),
        textAlign: TextAlign.center,
      ),
      const SizedBox(height: 24),
      ElevatedButton.icon(
        onPressed: _iniciarConexao,
        icon: const Icon(Icons.refresh),
        label: const Text('Tentar novamente'),
      ),
    ],
  ),
);
}

```

```

Widget _chipModo() {
  final modo = _controller.modosAtual;
  final texto = modo != null ? '${modo.width}x${modo.height}' : '—';
  return GestureDetector(
    onTap: _abrirSeletoresModo,
    child: Container(
      height: 40,
      padding: const EdgeInsets.symmetric(horizontal: 12),
      decoration: BoxDecoration(
        borderRadius: BorderRadius.circular(20),
        color: Colors.black45,
      ),
      child: Center(
        child: Text(
          texto,
          style: const TextStyle(color: Colors.white, fontSize: 13, fontWeight:
FontWeight.w600),
        ),
      ),
    ),
  );
}

```

```
}
```

```
Widget _botaoTopo(IconData icon, VoidCallback onTap, {bool ativo = false}) {  
  return GestureDetector(  
    onTap: onTap,  
    child: Container(  
      width: 40,  
      height: 40,  
      decoration: BoxDecoration(  
        shape: BoxShape.circle,  
        color: ativo ? Colors.redAccent : Colors.black45,  
      ),  
      child: Icon(icon, color: Colors.white, size: 20),  
    ),  
  );  
}
```

```
// =====  
// FIM DA PARTE 7  
// =====//  
=====//  
// PARTE 8: Painters (medição, desenho, grade, foco central)  
// =====
```

```
class _MedicaoPainter extends CustomPainter {  
  final Offset? a;  
  final Offset? b;  
  
  _MedicaoPainter(this.a, this.b);  
  
  @override  
  void paint(Canvas canvas, Size size) {  
    final paintLinha = Paint()  
      ..color = Colors.yellowAccent  
      ..strokeWidth = 2;  
    final paintPonto = Paint().color = Colors.yellowAccent;  
    if (a != null) canvas.drawCircle(a!, 6, paintPonto);  
    if (b != null) canvas.drawCircle(b!, 6, paintPonto);  
    if (a != null && b != null) canvas.drawLine(a!, b!, paintLinha);  
  }  
  
  @override  
  bool shouldRepaint(covariant _MedicaoPainter oldDelegate) {  
    return oldDelegate.a != a || oldDelegate.b != b;  
  }  
}
```

```
/// Desenha os traços à mão livre e os textos colocados pelo usuário sobre
```

```

/// a imagem da câmera.
class _DesenhoPainter extends CustomPainter {
  final List<List<Offset>> tracos;
  final List<_AnotacaoTexto> textos;

  _DesenhoPainter(this.tracos, this.textos);

  @override
  void paint(Canvas canvas, Size size) {
    final paintTraco = Paint()
      ..color = Colors.redAccent
      ..strokeWidth = 3.5
      ..strokeCap = StrokeCap.round
      ..style = PaintingStyle.stroke;

    for (final traco in tracos) {
      for (var i = 0; i < traco.length - 1; i++) {
        canvas.drawLine(traco[i], traco[i + 1], paintTraco);
      }
    }

    for (final anotacao in textos) {
      final textPainter = TextPainter(
        text: TextSpan(
          text: anotacao.texto,
          style: const TextStyle(
            color: Colors.white,
            fontSize: 16,
            fontWeight: FontWeight.w600,
            backgroundColor: Colors.black54,
          ),
        ),
        textDirection: TextDirection.ltr,
      )..layout();
      textPainter.paint(canvas, anotacao.posicao);
    }
  }

  @override
  bool shouldRepaint(covariant _DesenhoPainter oldDelegate) {
    // As listas são mutadas no lugar (add/clear), então comparar por
    // referência com "!=" nunca detectaria mudança. Sempre redesenha; o
    // custo é baixo e evita que traços/textos novos deixem de aparecer.
    return true;
  }
}

/// Desenha uma grade 3x3 (regra dos terços) sobre a imagem da câmera.

```

```

class _GradePainter extends CustomPainter {
  const _GradePainter();

  @override
  void paint(Canvas canvas, Size size) {
    final paint = Paint()
      ..color = Colors.white.withValues(alpha: 0.55)
      ..strokeWidth = 1;

    final tercoX1 = size.width / 3;
    final tercoX2 = size.width * 2 / 3;
    final tercoY1 = size.height / 3;
    final tercoY2 = size.height * 2 / 3;

    canvas.drawLine(Offset(tercoX1, 0), Offset(tercoX1, size.height), paint);
    canvas.drawLine(Offset(tercoX2, 0), Offset(tercoX2, size.height), paint);
    canvas.drawLine(Offset(0, tercoY1), Offset(size.width, tercoY1), paint);
    canvas.drawLine(Offset(0, tercoY2), Offset(size.width, tercoY2), paint);
  }

  @override
  bool shouldRepaint(covariant _GradePainter oldDelegate) => false;
}

/// Desenha uma mira de foco no centro da tela: círculo com cruz e cantos,
/// como referência visual para centralizar a amostra.
class _FocoCentralPainter extends CustomPainter {
  const _FocoCentralPainter();

  @override
  void paint(Canvas canvas, Size size) {
    final centro = Offset(size.width / 2, size.height / 2);
    final paint = Paint()
      ..color = Colors.greenAccent.withValues(alpha: 0.85)
      ..style = PaintingStyle.stroke
      ..strokeWidth = 1.6;

    const raio = 34.0;
    const tamanhoCruz = 10.0;
    const tamanhoCanto = 14.0;

    canvas.drawCircle(centro, raio, paint);

    // Cruz central
    canvas.drawLine(
      Offset(centro.dx - tamanhoCruz, centro.dy),
      Offset(centro.dx + tamanhoCruz, centro.dy),
      paint,

```

```

);
canvas.drawLine(
    Offset(centro.dx, centro.dy - tamanhoCruz),
    Offset(centro.dx, centro.dy + tamanhoCruz),
    paint,
);

// Cantos, como uma mira de foco de câmera
final box = Rect.fromCircle(center: centro, radius: raio + 18);
void desenharCanto(Offset ponto, Offset horizontal, Offset vertical) {
    canvas.drawLine(ponto, ponto + horizontal, paint);
    canvas.drawLine(ponto, ponto + vertical, paint);
}

desenharCanto(box.topLeft, const Offset(tamanhoCanto, 0), const Offset(0,
tamanhoCanto));
desenharCanto(box.topRight, const Offset(-tamanhoCanto, 0), const Offset(0,
tamanhoCanto));
desenharCanto(box.bottomLeft, const Offset(tamanhoCanto, 0), const Offset(0,
-tamanhoCanto));
desenharCanto(box.bottomRight, const Offset(-tamanhoCanto, 0), const Offset(0,
-tamanhoCanto));
}

@override
bool shouldRepaint(covariant _FocoCentralPainter oldDelegate) => false;
}

// =====
// FIM DA PARTE 8 – CÓDIGO COMPLETO
// =====

```